Kiểm thử phần mềm nhập môn

Tài liệu gốc kèm thực hành bổ sung · 29/10/2026

Mục lục

- Chương 1: Nền tảng kiểm thử: Mục tiêu và các mức kiểm thử
- Chương 1: Nền tảng kiểm thử: Thiết kế test case
- Chương 2: Kỹ thuật kiểm thử và xử lý lỗi: Phân vùng tương đương và giá trị biên
- Chương 2: Kỹ thuật kiểm thử và xử lý lỗi: Báo cáo lỗi và kiểm thử hồi quy
- Chương 3: Kiểm thử trong vòng đời phát triển: SDLC, Agile và chiến lược kiểm thử
- Chương 3: Kiểm thử trong vòng đời phát triển: Review yêu cầu và truy vết kiểm thử
- Chương 4: Thiết kế kiểm thử theo hành vi: Bảng quyết định và tổ hợp điều kiện
- Chương 4: Thiết kế kiểm thử theo hành vi: Kiểm thử chuyển trạng thái
- Chương 5: Kiểm thử dữ liệu và dịch vụ: Kiểm thử API và phân quyền
- Chương 5: Kiểm thử dữ liệu và dịch vụ: Kiểm thử cơ sở dữ liệu và tính toàn vẹn
- Chương 6: Chất lượng và thực hành tổng hợp: Hiệu năng, khả dụng và tương thích
- Chương 6: Chất lượng và thực hành tổng hợp: Dự án nhỏ: kế hoạch kiểm thử và tự động hóa

Mục tiêu và các mức kiểm thử

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Kiểm thử phần mềm nhập môn.
- Chương 1: Nền tảng kiểm thử.
- Các bài trong chương: Mục tiêu và các mức kiểm thử; Thiết kế test case.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Giải thích giá trị kiểm thử và phân biệt các mức kiểm thử phổ biến.

Kiến thức nền

Kiểm thử cung cấp thông tin về chất lượng và rủi ro của phần mềm. Nó có thể phát hiện lỗi nhưng một tập kiểm thử hữu hạn không chứng minh chương trình không còn lỗi. Kiểm thử nên bám yêu cầu, ngữ cảnh sử dụng và mức độ ảnh hưởng của sai sót.

Unit test tập trung vào đơn vị nhỏ như hàm hoặc lớp; integration test kiểm tra tương tác giữa các thành phần; system test kiểm tra hệ thống hoàn chỉnh; acceptance test đánh giá khả năng đáp ứng nhu cầu chấp nhận đã thống nhất. Ranh giới cụ thể phụ thuộc thiết kế hệ thống.

Kiểm thử tĩnh xem xét sản phẩm công việc mà không chạy phần mềm, như review yêu cầu hoặc mã nguồn. Kiểm thử động thực thi phần mềm và quan sát kết quả. Hai cách bổ sung cho nhau.

1. Chất lượng, lỗi và bằng chứng

Một sai sót của con người có thể tạo khuyết tật trong tài liệu hoặc mã; khuyết tật khi được kích hoạt có thể gây thất bại quan sát được. Không phải mọi khuyết tật đều biểu hiện trong mọi lần chạy. Kiểm thử cần mô tả điều kiện kích hoạt và kết quả thay vì chỉ ghi "có lỗi".

Verification thường xem sản phẩm công việc có đáp ứng yêu cầu đã chỉ định; validation xem giải pháp có phù hợp nhu cầu sử dụng trong ngữ cảnh. Hai góc nhìn bổ sung nhau. Số lỗi tìm được không trực tiếp đo năng lực tester hoặc chất lượng toàn hệ thống.

2. Phân biệt mức và loại

```
Mức kiểm thử        Ví dụ LMS
Unit                Hàm tính phần trăm và xử lý tổng câu bằng 0
Integration         Service ghi danh phối hợp database và kiểm tra quyền
```

```
System              Luồng đăng ký -> mở bài -> nộp quiz trên hệ thống
Acceptance          Đối chiếu tiêu chí học viên bắt đầu học sau ghi danh
```

Functional/non-functional là cách phân loại mục tiêu, không phải thay thế các mức trên. Một integration test vẫn có thể đo thuộc tính phi chức năng.

3. Thiết kế phép kiểm tra có ý nghĩa

Với tỷ lệ đúng, test cần đầu vào và kỳ vọng: 2/5 thành 40%; tổng 0 phải theo hợp đồng. Không tự chọn 0% nếu yêu cầu muốn "chưa có dữ liệu". Ví dụ này cho thấy test phải bám ý nghĩa sản phẩm, không chỉ biểu thức toán.

4. Bài tập và lời giải

Viết bốn test ở bốn mức cho nộp quiz. Unit kiểm tra chấm một câu; integration kiểm tra lưu kết quả và câu trả lời cùng giao dịch; system kiểm tra hiển thị sau nộp; acceptance kiểm tra học viên nhận được hướng ôn theo tiêu chí. Nêu cách dùng dữ liệu lab và không tự tạo điểm cho học viên thật.

> **Bài tập vận dụng bổ sung**
>
> với website học trực tuyến, đề xuất một unit test cho hàm tính điểm, một integration test giữa ghi danh và quyền học, một acceptance test cho quy trình học viên nộp quiz.

Câu hỏi tự kiểm tra

1. Mục tiêu phù hợp của kiểm thử là gì?
2. Kiểm tra riêng một hàm tính điểm thường thuộc mức nào?
3. Kiểm tra tương tác giữa API và cơ sở dữ liệu thường là gì?
4. Acceptance test tập trung vào điều gì?
5. Review mã nguồn mà không chạy chương trình là gì?

Gợi ý đáp án và điểm cần nhớ

- Cung cấp thông tin chất lượng và phát hiện vấn đề. Kiểm thử giảm bất định nhưng không chứng minh tuyệt đối phần mềm không có lỗi.
- Unit test. Unit test tập trung vào một đơn vị nhỏ.
- Integration test. Integration test kiểm tra sự phối hợp giữa các thành phần.
- Tiêu chí chấp nhận và nhu cầu đã thống nhất. Acceptance test đánh giá khả năng đáp ứng nhu cầu sử dụng theo tiêu chí chấp nhận.
- Kiểm thử tĩnh. Hoạt động tĩnh không yêu cầu thực thi phần mềm.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.

- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Mục tiêu và các mức kiểm thử — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: unit test xanh nhưng người dùng không đăng nhập được. Unit, tích hợp và đầu cuối kiểm tra những ranh giới khác nhau; một mức không tự thay thế các mức còn lại. Chọn mức theo rủi ro và chi phí phản hồi.

Tự luyện: phân loại kiểm tra hàm tính điểm, API nộp bài và form nộp bài trong trình duyệt. **Hướng giải:** nêu phụ thuộc được dùng thật hay thay thế; kiểm tra quyền và dữ liệu lưu khi cần. Số test lớn không tự chứng minh đã phủ yêu cầu quan trọng.

Nguồn học tiếp

https://docs.pytest.org/en/stable/how-to/assert.html

KIỂM THỬ PHẦN MỀM NHẬP MÔN

Chương 1: Nền tảng kiểm thử · Bài 2

Thiết kế test case

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Kiểm thử phần mềm nhập môn.
- Chương 1: Nền tảng kiểm thử.
- Các bài trong chương: Mục tiêu và các mức kiểm thử; Thiết kế test case.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Viết test case có thể lặp lại và đánh giá được.

Kiến thức nền

Một test case nên có mục tiêu, điều kiện trước, dữ liệu đầu vào, các bước và kết quả mong đợi. Kết quả mong đợi cần cụ thể theo yêu cầu; câu như "hệ thống chạy tốt" không đủ để xác định pass hay fail.

Phân biệt kết quả mong đợi với kết quả thực tế. Khi chạy test, ghi môi trường và phiên bản để có thể tái hiện. Dữ liệu thử cần kiểm soát, không dùng tài khoản hoặc giao dịch thật nếu có môi trường kiểm thử riêng.

Bao phủ cả luồng hợp lệ và các tình huống không hợp lệ: thiếu trường bắt buộc, sai kiểu, không đủ quyền hoặc tài nguyên không tồn tại. Không chỉ kiểm tra giao diện báo thành công mà bỏ qua việc dữ liệu thực sự được lưu đúng.

1. Mẫu test case có thể thực hiện

```
ID: ENR-001
Mục tiêu: ghi danh khóa miễn phí một lần
Điều kiện trước: tài khoản lab hoạt động, khóa published, chưa ghi danh
Dữ liệu: user U1, course C1, price=0
Bước 1: mở danh sách khóa, chọn C1
Bước 2: bấm Đăng ký, chờ request hoàn tất
Kỳ vọng: một phản hồi thành công, một ghi danh, khóa có trong Khóa học của tôi
Sau test: chỉ dọn dữ liệu do ca này tạo trong database test
```

Kết quả thực tế và bằng chứng chỉ điền sau khi chạy. Không điền "pass" sẵn trong mẫu để minh họa như một kết quả thật.

2. Bao phủ tình huống thay vì nhân bản thao tác

Test dương kiểm tra đăng ký hợp lệ. Test âm kiểm tra khóa không tồn tại, chưa xác thực, khóa không cho đăng ký hoặc đầu vào sai kiểu theo hợp đồng. Test lặp/đồng thời kiểm tra tính duy nhất. Một bộ test chỉ đổi tên tài khoản nhưng đi cùng đường thành công không bao phủ các rủi ro này.

3. Kết quả mong đợi đủ cụ thể

"Đăng ký tốt" không kiểm chứng rõ. "API phản hồi thành công theo hợp đồng; bảng ghi danh tăng đúng một bản ghi U1–C1; không phát sinh thanh toán cho khóa miễn phí" là các quan sát cụ thể. Nếu không có quyền xem database, dùng API kiểm tra hoặc cơ chế bằng chứng được nhóm cho phép, không bỏ qua xác minh.

4. Bài tập có đáp án định hướng

Viết ENR-002 cho bấm lặp và ENR-003 cho không đủ quyền. Phân biệt luồng bấm ở UI với hai request thật: khóa nút chỉ là một lớp chống lặp. Đáp án cần kiểm tra ràng buộc server và không tạo bản ghi thứ hai. Ghi rõ mã lỗi dự kiến theo hợp đồng hiện tại, không đoán mọi dự án đều trả 409.

> **Bài tập vận dụng bổ sung**
>
> viết test case cho học viên chưa ghi danh mở bài có giới hạn. Ghi điều kiện trước, bước truy cập, kết quả về quyền và xác nhận không phát sinh tiến độ học trái phép.

Câu hỏi tự kiểm tra

1. Thành phần nào giúp quyết định test pass hay fail?
2. Điều kiện trước mô tả điều gì?
3. Test đăng nhập chỉ dùng mật khẩu đúng đã đủ chưa?
4. Kết quả mong đợi nào kiểm chứng được rõ hơn?
5. Vì sao ghi phiên bản và môi trường khi chạy test?

Gợi ý đáp án và điểm cần nhớ

- Kết quả mong đợi cụ thể. Cần đối chiếu kết quả thực tế với kết quả mong đợi.
- Trạng thái cần có trước khi thực hiện test. Ví dụ tài khoản đã ghi danh là một điều kiện trước.
- Chưa, cần tình huống sai và dữ liệu không hợp lệ phù hợp. Luồng không hợp lệ có thể bộc lộ lỗi xác thực và xử lý đầu vào.
- API trả 403 và không tạo bản ghi tiến độ. Mã phản hồi và trạng thái dữ liệu là các quan sát cụ thể.
- Giúp tái hiện và phân tích kết quả. Hành vi có thể khác theo cấu hình hoặc phiên bản.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Thiết kế test case — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: test case chỉ ghi "đăng nhập thành công". Cần điều kiện đầu, dữ liệu, bước hành động, kết quả quan sát được và cách dọn. Kết quả mong đợi phải đủ cụ thể để hai người chấm giống nhau.

Tự luyện: viết ca đăng nhập đúng, sai mật khẩu và tài khoản bị khóa. **Hướng giải:** nêu trang/response, trạng thái phiên và quyền sau tải lại; không ghi mật khẩu thật vào báo cáo. "Không crash" chưa đủ chứng minh nghiệp vụ đúng.

Nguồn học tiếp

https://docs.pytest.org/en/stable/how-to/assert.html

Phân vùng tương đương và giá trị biên

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Kiểm thử phần mềm nhập môn.
- Chương 2: Kỹ thuật kiểm thử và xử lý lỗi.
- Các bài trong chương: Phân vùng tương đương và giá trị biên; Báo cáo lỗi và kiểm thử hồi quy.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

Mục tiêu bài học

Chọn dữ liệu đại diện thay vì kiểm tra ngẫu nhiên mọi giá trị.

Kiến thức nền

Phân vùng tương đương chia miền dữ liệu thành các nhóm dự kiến được xử lý tương tự, rồi chọn đại diện cho mỗi nhóm. Phải xét cả nhóm hợp lệ và không hợp lệ theo yêu cầu; đây là kỹ thuật lựa chọn test, không phải chứng minh mọi giá trị trong nhóm chắc chắn có hành vi giống nhau.

Với trường tuổi chỉ chấp nhận số nguyên từ 18 tới 60, có các vùng nhỏ hơn 18, từ 18 đến 60 và lớn hơn 60. Có thể chọn 17, 30, 61 làm đại diện. Kiểu đầu vào không phải số nguyên cũng cần xem xét theo hợp đồng dữ liệu.

Phân tích giá trị biên tập trung vào ranh giới vì lỗi so sánh và lệch một đơn vị thường xuất hiện ở đó. Với ví dụ trên, các điểm sát biên như 17,18,19 và 59,60,61 giúp kiểm tra cách xử lý ngưỡng dưới và trên.

1. Bắt đầu từ miền đầu vào

Giả sử tuổi bắt buộc là số nguyên, hợp lệ 18–60 bao gồm hai đầu. Chia miền số nguyên thành dưới 18, 18–60, trên 60; xét thêm dữ liệu thiếu, chuỗi không chuyển được hoặc số thập phân theo API. Không dùng kỹ thuật biên để né việc làm rõ kiểu dữ liệu.

2. Ví dụ dữ liệu và kỳ vọng

```
Giá trị    Phân loại             Kỳ vọng theo giả định
17         Ngay dưới biên dưới   Từ chối
18         Tại biên dưới         Chấp nhận
19         Ngay trên biên dưới   Chấp nhận
59         Ngay dưới biên trên   Chấp nhận
```

```
60          Tại biên trên         Chấp nhận
61          Ngay trên biên trên   Từ chối
```

Đây là lựa chọn ba điểm quanh mỗi biên cho dữ liệu rời rạc. Tài liệu và tiêu chuẩn có thể định nghĩa các biến thể bao phủ khác; luôn ghi biến thể đang dùng. Với số thực không có "số kế tiếp" giống số nguyên nếu chưa quy định độ chính xác và bước dữ liệu.

3. Sai lầm thường gặp

Chỉ thử 30 rồi kết luận 18 và 60 đúng; chỉ thử giá trị hợp lệ; tự chấp nhận chuỗi "18" dù hợp đồng yêu cầu JSON number; đưa null và chuỗi rỗng vào cùng vùng dù server xử lý khác. Những quyết định này có thể làm mất ca quan trọng.

4. Bài tập và lời giải

Điểm quiz là số nguyên 0–100: thử -1,0,1,99,100,101, cùng trường thiếu và sai kiểu theo yêu cầu. Nếu số lượt tối đa là 3, kiểm tra trước lượt 3, sau lượt 3 và trường hợp lượt cuối đang làm: đây còn là bài toán trạng thái, không chỉ biên số. Nêu giới hạn của kỹ thuật thay vì tuyên bố sáu giá trị đã bao phủ mọi lỗi.

Bài tập vận dụng bổ sung

trường điểm nhận số nguyên 0–100. Liệt kê vùng hợp lệ, các vùng không hợp lệ và dữ liệu biên; nêu kết quả mong đợi cho từng giá trị.

Câu hỏi tự kiểm tra

1. Phân vùng tương đương giúp làm gì?
2. Tuổi hợp lệ là số nguyên 18–60. Giá trị nào thuộc vùng hợp lệ?
3. Bộ nào kiểm tra sát biên dưới 18 cho đầu vào nguyên?
4. Điểm hợp lệ từ 0 đến 100. Giá trị 101 thuộc vùng nào?
5. Vì sao cần kiểm tra giá trị biên?

Gợi ý đáp án và điểm cần nhớ

- Chọn đại diện cho các nhóm đầu vào xử lý tương tự. Chọn đại diện giúp giảm số lượng test có chủ đích.
- 30. 30 nằm trong khoảng bao gồm cả 18 và 60.
- 17, 18, 19. Các điểm ngay dưới, tại và ngay trên biên kiểm tra điều kiện ngưỡng.
- Không hợp lệ phía trên. 101 vượt giới hạn trên; không tự suy đoán hành vi làm tròn.
- Dễ xuất hiện lỗi điều kiện và lệch một đơn vị. Kiểm tra biên nhắm vào một nhóm lỗi phổ biến, không bao phủ mọi lỗi.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Phân vùng tương đương và giá trị biên — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: điểm hợp lệ 0..10 nhưng chỉ thử 5. Chọn miền hợp lệ, dưới miền, trên miền và dữ liệu không phải số; thử sát biên theo kiểu dữ liệu được hỗ trợ.

```
def valid_score(value):
    return isinstance(value, (int, float)) and not isinstance(value, bool) and 0 <= value <= 10

assert valid_score(0) and valid_score(10)
assert not valid_score(-1) and not valid_score(11)
assert not valid_score("5")
```

Tự luyện: làm rõ có nhận chuỗi số và NaN không. **Hướng giải:** xác định hợp đồng trước khi kết luận test fail; đổi kiểu dữ liệu có thể thay đổi tập biên.

Nguồn học tiếp

https://docs.pytest.org/en/stable/how-to/assert.html

Báo cáo lỗi và kiểm thử hồi quy

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Kiểm thử phần mềm nhập môn.
- Chương 2: Kỹ thuật kiểm thử và xử lý lỗi.
- Các bài trong chương: Phân vùng tương đương và giá trị biên; Báo cáo lỗi và kiểm thử hồi quy.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Báo lỗi đủ thông tin và xác nhận bản sửa một cách có căn cứ.

Kiến thức nền

Báo cáo lỗi nên có tiêu đề ngắn, môi trường, điều kiện trước, bước tái hiện, kết quả thực tế, kết quả mong đợi và bằng chứng liên quan. Không đưa mật khẩu hoặc token vào ảnh chụp và nhật ký chia sẻ.

Severity mô tả mức độ ảnh hưởng của lỗi; priority mô tả thứ tự cần xử lý theo quyết định của nhóm. Hai yếu tố liên quan nhưng không đồng nhất: một lỗi giao diện nhỏ trên trang chiến dịch sắp ra mắt có thể được ưu tiên sớm.

Sau sửa lỗi, confirmation testing kiểm tra lại lỗi cụ thể đã được sửa. Regression testing kiểm tra các hành vi liên quan để tìm tác động phụ ngoài ý muốn. Test tự động có ích cho tình huống lặp lại nhưng vẫn cần dữ liệu và kết quả mong đợi đúng.

1. Mẫu báo lỗi có bằng chứng

```
ID: BUG-LAB-01 (tình huống minh họa)
Tiêu đề: Hai yêu cầu ghi danh cùng khóa tạo hai bản ghi
Môi trường: database test, phiên bản được ghi cụ thể
Điều kiện: U1 chưa ghi danh C1
Bước tái hiện: gửi hai yêu cầu đồng thời với dữ liệu test hợp lệ
Thực tế: mô tả quan sát sau khi chạy, không điền trước
Mong đợi: chỉ một ghi danh theo quy tắc duy nhất
Bằng chứng: ID request, trạng thái, ảnh/query đã che bí mật
```

Một mô tả "bấm thấy sai" thiếu thời điểm, dữ liệu và kết quả nên khó tái hiện. Báo cáo cần đủ chi tiết nhưng không đưa token, mật khẩu hoặc dữ liệu cá nhân không cần thiết.

2. Severity và priority

Severity đánh giá ảnh hưởng kỹ thuật/nghiệp vụ; priority là mức ưu tiên xử lý trong bối cảnh phát hành. Lỗi giao diện nhỏ có thể được ưu tiên cao do trang sắp ra mắt; không đồng nhất hai trường. Dùng thang và quy ước của nhóm thay vì tự gắn mọi lỗi là nghiêm trọng nhất.

3. Kiểm tra sau sửa

Confirmation testing thực hiện lại điều kiện gây lỗi để xác nhận bản sửa. Regression testing mở rộng sang các hành vi có nguy cơ bị ảnh hưởng: đăng ký lần đầu, trùng, khóa có phí, quyền mở bài và danh sách khóa. Chọn phạm vi theo thay đổi, không mặc định chạy mọi test vô hạn.

4. Bài tập có lời giải định hướng

Bản sửa thêm unique constraint nhưng UI báo lỗi thô khi request thứ hai tới. Cần báo vấn đề xử lý phản hồi riêng, không kết luận sửa database thất bại nếu tính duy nhất đã đúng. Nộp một bug report, một ca xác nhận và ít nhất bốn ca hồi quy có lý do lựa chọn; không dùng dữ liệu thật để cố tình gây lỗi.

> **Bài tập vận dụng bổ sung**
>
> báo lỗi “học viên hết lượt vẫn mở được quiz”. Viết bước tái hiện, kiểm tra trạng thái dữ liệu và đề xuất các test hồi quy cho trường hợp còn lượt và tiếp tục lượt đang làm.

Câu hỏi tự kiểm tra

1. Báo cáo lỗi cần thông tin nào để tái hiện?
2. Severity thường thể hiện điều gì?
3. Kiểm tra lại chính lỗi vừa sửa gọi là gì?
4. Kiểm thử hồi quy nhằm mục đích gì?
5. Thông tin nào nên che khi chia sẻ bằng chứng lỗi?

Gợi ý đáp án và điểm cần nhớ

- Điều kiện trước và các bước thực hiện. Các bước và điều kiện trước giúp người khác tái hiện lỗi.
- Mức độ ảnh hưởng của lỗi. Severity đánh giá ảnh hưởng; priority đánh giá mức ưu tiên xử lý.
- Confirmation testing. Confirmation testing xác nhận lỗi cụ thể được khắc phục.
- Phát hiện tác động phụ tới hành vi đã hoạt động. Thay đổi có thể ảnh hưởng chức năng khác nên cần kiểm tra hồi quy phù hợp.
- Mật khẩu và token truy cập. Thông tin xác thực không được đưa vào bằng chứng chia sẻ.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.

- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Báo cáo lỗi và kiểm thử hồi quy — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: lỗi "thỉnh thoảng" không có môi trường tái hiện. Báo cáo cần bước tối thiểu, dữ liệu giả lập, mong đợi/thực tế, phiên bản và bằng chứng đã loại thông tin nhạy cảm.

Tự luyện: mô tả một lỗi giả định rõ nhãn giả định, sau đó viết ca hồi quy. **Hướng giải:** phân biệt retest lỗi vừa sửa với hồi quy khu vực liên quan. Nếu script kiểm thử sai selector, ghi lỗi công cụ và thử lại; không báo thành lỗi ứng dụng, không xóa nhật ký lần đầu để làm báo cáo đẹp.

Nguồn học tiếp

https://docs.pytest.org/en/stable/how-to/assert.html

SDLC, Agile và chiến lược kiểm thử

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Kiểm thử phần mềm nhập môn.
- Chương 3: Kiểm thử trong vòng đời phát triển.
- Các bài trong chương: SDLC, Agile và chiến lược kiểm thử; Review yêu cầu và truy vết kiểm thử.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Chọn hoạt động kiểm thử phù hợp giai đoạn phát triển và rủi ro sản phẩm.

Kiến thức nền

SDLC mô tả hoạt động phát triển từ hiểu nhu cầu tới vận hành. Kiểm thử nên tham gia sớm qua review yêu cầu và tiêu chí chấp nhận, không chờ mã nguồn hoàn thành mới bắt đầu. Phát hiện yêu cầu mơ hồ sớm có thể tránh xây sai chức năng.

Trong phát triển lặp, kiểm thử được thực hiện xuyên suốt từng vòng. Definition of Done là thỏa thuận về điều kiện hoàn tất, khác tiêu chí chấp nhận cụ thể cho từng câu chuyện. Chất lượng là trách nhiệm chung; có tester không có nghĩa lập trình viên không cần kiểm tra.

Chiến lược chọn phạm vi, mức kiểm thử và mức tự động hóa theo rủi ro. Luồng thanh toán hoặc phân quyền có tác động lớn nên cần được ưu tiên hợp lý. Không thể kiểm thử mọi kết hợp vô hạn; cần ghi giả định, phần đã kiểm tra và rủi ro còn lại. Mô hình phát triển ảnh hưởng cách tổ chức nhưng không xóa nhu cầu có kết quả mong đợi.

1. Từ yêu cầu tới vận hành

Các hoạt động hiểu nhu cầu, thiết kế, triển khai, kiểm thử và vận hành không nhất thiết diễn ra một lần theo đường thẳng. Trong vòng lặp, thông tin phản hồi cập nhật sản phẩm và bộ test. Kiểm thử sớm có thể là review mô tả nghiệp vụ trước khi có giao diện.

2. Ví dụ kế hoạch cho một vòng phát triển

```
Yêu cầu: khóa miễn phí chỉ ghi danh một lần
Trước code: review quyền, trạng thái khóa và thông báo lỗi
Trong code: unit cho quy tắc; integration cho lưu và chống trùng
```

```
Sau tích hợp: UI cho trạng thái chờ, lỗi, thành công và màn hình hẹp
Trước phát hành: hồi quy theo tác động, báo cáo giới hạn và rủi ro
```

Acceptance criteria mô tả điều kiện riêng của yêu cầu. Definition of Done là thỏa thuận chung về mức hoàn tất. Một câu chuyện có tiêu chí đạt nhưng chưa hoàn tất các yêu cầu chất lượng chung vẫn có thể chưa “done”.

3. Chọn phạm vi theo rủi ro

Ước lượng khả năng sai và tác động, xác định vùng ưu tiên, liên kết từng vùng tới test. Điểm số rủi ro chỉ là công cụ thảo luận; không tạo cảm giác chính xác tuyệt đối nếu đầu vào chỉ là đánh giá sơ bộ. Dành thời gian cho lỗi đã thấy và các ràng buộc quyền/dữ liệu quan trọng.

4. Bài tập và đáp án

Trong hai ngày, nhóm sửa luồng nộp quiz và đổi màu tiêu đề. Ưu tiên tính điểm, lưu kết quả và lượt làm trước phần màu vì ảnh hưởng cao hơn; vẫn kiểm tra hiển thị cơ bản. Nộp kế hoạch ai làm gì, dữ liệu nào, tiêu chí vào/ra và phần chưa thể kiểm tra. Không gọi kế hoạch là bằng chứng đã chạy test.

> **Bài tập vận dụng bổ sung**
>
> Viết tiêu chí chấp nhận cho đăng ký khóa, đề xuất hoạt động review, unit, integration và system trong một vòng phát triển. Nêu ba rủi ro cần ưu tiên.

Hướng giải bài tập bổ sung

Phân biệt điều kiện hoàn tất chung với tiêu chí riêng của chức năng. Bao phủ tính đúng dữ liệu và quyền, không chỉ màu thông báo.

Câu hỏi tự kiểm tra

1. Kiểm thử nên bắt đầu khi nào?
2. Trong Agile, trách nhiệm chất lượng thuộc về ai?
3. Tiêu chí chấp nhận mô tả điều gì?
4. Khi thời gian có hạn, nên ưu tiên test theo gì?
5. Có thể kiểm tra hết mọi kết hợp đầu vào của mọi hệ thống không?

Gợi ý đáp án và điểm cần nhớ

- Từ sớm, gồm review yêu cầu. Review sớm giúp phát hiện vấn đề trước khi viết mã.
- Cả nhóm theo vai trò phối hợp. Chất lượng không được giao hoàn toàn cho một vai trò.
- Điều kiện cụ thể chức năng cần đáp ứng. Tiêu chí giúp đánh giá hành vi theo nhu cầu.
- Rủi ro và tác động nghiệp vụ. Ưu tiên dựa trên xác suất và ảnh hưởng sai sót.

- Thường không, cần chọn phạm vi có căn cứ. Kiểm thử toàn bộ thường không khả thi với hệ thống thực tế.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

SDLC, Agile và chiến lược kiểm thử — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: nhận tính năng quá muộn mới nghĩ dữ liệu kiểm thử. Xem yêu cầu sớm, làm rõ tiêu chí chấp nhận và khả năng quan sát; phân phối kiểm thử suốt quá trình thay đổi.

Tự luyện: lập chiến lược một trang cho ghi danh khóa học: phạm vi, rủi ro, mức kiểm tra, dữ liệu và điều kiện kết thúc. **Hướng giải:** ưu tiên quyền, thanh toán và dữ liệu học tập; nêu phần ngoài phạm vi. Không dùng Agile để bỏ tài liệu hoặc dùng "đã test" không có bằng chứng làm điều kiện phát hành.

Nguồn học tiếp

https://docs.pytest.org/en/stable/how-to/assert.html

Review yêu cầu và truy vết kiểm thử

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Kiểm thử phần mềm nhập môn.
- Chương 3: Kiểm thử trong vòng đời phát triển.
- Các bài trong chương: SDLC, Agile và chiến lược kiểm thử; Review yêu cầu và truy vết kiểm thử.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Phát hiện yêu cầu mơ hồ và liên kết test với mục tiêu nghiệp vụ.

Kiến thức nền

Review tĩnh có thể áp dụng cho yêu cầu, thiết kế, test case và mã nguồn. Một yêu cầu tốt cần rõ ràng, nhất quán và có thể kiểm chứng. "Trang phải nhanh" cần ngữ cảnh đo như thao tác, tải, môi trường và ngưỡng trước khi chuyển thành kiểm thử hiệu năng.

Truy vết nối yêu cầu với test và kết quả hoặc lỗi liên quan. Ma trận có thể đơn giản: mã yêu cầu, ca kiểm thử, trạng thái, mã lỗi. Truy vết giúp nhìn phần chưa có test và xác định test bị ảnh hưởng khi yêu cầu đổi.

Bao phủ yêu cầu không chứng minh bao phủ tất cả rủi ro. Một test gắn mã yêu cầu nhưng không kiểm tra điều kiện quan trọng vẫn có thể bỏ sót lỗi. Review nên tập trung sản phẩm công việc, dùng ví dụ cụ thể và ghi kết luận; tránh biến thành đánh giá cá nhân.

1. Checklist review yêu cầu

- Rõ chủ thể, hành động và tài nguyên không?
- Có điều kiện trước, biên và trường hợp lỗi không?
- Thuật ngữ có nhất quán với màn hình và API không?
- Có thể quan sát kết quả bằng quyền và công cụ hiện có không?
- Có xung đột giữa hai yêu cầu hoặc thiếu quyết định nghiệp vụ không?

Ví dụ "người dùng xem được kết quả" cần làm rõ kết quả của ai, vai trò nào, thời điểm nào và có che đáp án trước lượt cuối không.

2. Ma trận truy vết tối thiểu

```
Yêu cầu   Test              Mục tiêu                Kết quả/Lỗi
REQ-01    RES-OWN           Xem bài của chính mình  Điền sau chạy
REQ-01    RES-OTHER         Chặn bài người khác     Điền sau chạy
REQ-02    RES-HIDE          Che đáp án khi còn lượt Điền sau chạy
```

Không lấy việc một ô đã có tên test làm chứng cứ test đầy đủ. Kiểm tra xem kỳ vọng có thực sự bám điều kiện của yêu cầu hay chỉ gắn mã để hoàn thành bảng.

3. Xử lý yêu cầu đổi

Nếu đổi quy tắc hiện đáp án, tìm test liên quan lịch sử, nộp bài và quyền giảng viên. Cập nhật kỳ vọng có chủ ý và giữ lịch sử phiên bản. Không chỉnh test chỉ để khớp hành vi hiện tại khi hành vi đó chưa được xác nhận đúng.

4. Bài tập và lời giải

Viết lại "gợi ý phải chính xác" thành tiêu chí quan sát: đúng phần kiến thức, số câu đúng/tổng, ngưỡng, thiếu dữ liệu và link hợp lệ. Chọn một tiêu chí còn mơ hồ để ghi câu hỏi cho người phụ trách. Đáp án phải nêu được đầu vào minh họa, kỳ vọng cụ thể và giới hạn, ví dụ chưa đủ 5 câu không kết luận yếu.

> **Bài tập vận dụng bổ sung**
>
> Viết lại yêu cầu "đăng ký dễ dàng" thành hành vi quan sát được. Lập bảng ba yêu cầu với ít nhất hai test mỗi yêu cầu và chỉ rõ bằng chứng cần thu thập.

Hướng giải bài tập bổ sung

Có test cho đường thành công, lỗi và ràng buộc dữ liệu. Không dùng tỉ lệ test chạy qua làm bằng chứng duy nhất về mức bao phủ.

Câu hỏi tự kiểm tra

1. Review tĩnh có bắt buộc chạy phần mềm không?
2. Yêu cầu nào cần làm rõ trước khi kiểm thử?
3. Truy vết giúp điều gì khi yêu cầu thay đổi?
4. Một yêu cầu gắn đúng một test có luôn đủ bao phủ không?
5. Review nên tập trung vào đâu?

Gợi ý đáp án và điểm cần nhớ

- Không. Review sản phẩm công việc có thể diễn ra trước thực thi.
- Trang phải nhanh. Từ nhanh thiếu ngữ cảnh và tiêu chí đo.

- Tìm các test có thể bị ảnh hưởng. Liên kết yêu cầu và test hỗ trợ phân tích tác động.
- Không. Một yêu cầu có thể có nhiều nhánh và điều kiện cần kiểm tra.
- Sản phẩm công việc và bằng chứng cụ thể. Mục đích là cải thiện sản phẩm và phát hiện vấn đề.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Review yêu cầu và truy vết kiểm thử — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: yêu cầu "quiz hoàn thành" chưa rõ là đã mở, đã nộp hay đạt. Làm rõ trạng thái và quy tắc điểm trước khi tạo test. Truy vết nối yêu cầu → ca → kết quả → lỗi, không chỉ đếm dòng trong bảng.

Tự luyện: viết tiêu chí cho chưa bắt đầu, đang làm, hết giờ chưa nộp và đã nộp. **Hướng giải:** đạt/chưa đạt là thuộc tính kết quả, không thay thế trạng thái lượt. Nếu mở rồi bỏ dở vẫn tốn lượt theo yêu cầu, test phải xác nhận quy tắc đó thay vì mặc định nộp mới tính lượt.

Nguồn học tiếp

https://docs.pytest.org/en/stable/how-to/assert.html

Bảng quyết định và tổ hợp điều kiện

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Kiểm thử phần mềm nhập môn.
- Chương 4: Thiết kế kiểm thử theo hành vi.
- Các bài trong chương: Bảng quyết định và tổ hợp điều kiện; Kiểm thử chuyển trạng thái.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Chuyển quy tắc nghiệp vụ nhiều điều kiện thành các ca kiểm thử rõ ràng.

Kiến thức nền

Bảng quyết định biểu diễn điều kiện và hành động theo từng quy tắc. Hữu ích khi kết quả phụ thuộc nhiều điều kiện kết hợp. Mỗi cột có thể mô tả một tổ hợp cùng hành vi mong đợi; cần phân biệt tổ hợp không thể xảy ra với tổ hợp chưa được kiểm tra.

Ví dụ quyền học phụ thuộc tài khoản hợp lệ, trạng thái ghi danh và nội dung miễn phí. Đặt giả định rõ trước khi lập bảng. Không mặc định mọi ứng dụng có cùng thứ tự ưu tiên giữa các điều kiện; cần hỏi hoặc xác nhận yêu cầu nếu quy tắc xung đột.

Khi số điều kiện tăng, số tổ hợp tăng nhanh. Có thể nhóm các quy tắc tương đương hoặc chọn kỹ thuật giảm tổ hợp theo rủi ro, nhưng phải ghi giới hạn. Pairwise không bảo đảm tìm mọi lỗi cần ba điều kiện tương tác. Dữ liệu test phải thực sự tạo được trạng thái đã mô tả.

1. Xác định điều kiện độc lập

Giả định bài miễn phí được xem không cần ghi danh; bài giới hạn cần ghi danh hợp lệ. Trong phạm vi tài khoản được phép dùng hệ thống, hai điều kiện Free và Enrolled cho bảng dưới. Nếu tài khoản bị khóa có quy tắc riêng, phải thêm hoặc làm rõ trước, không âm thầm áp bảng cho mọi vai trò.

Quy tắc	Free	Enrolled	Cho xem
R1	Có	Có	Có
R2	Có	Không	Có
R3	Không	Có	Có
R4	Không	Không	Không

Có thể gộp R1 và R2 khi Enrolled không ảnh hưởng kết quả trong trường hợp Free; việc rút gọn cần giữ ý nghĩa điều kiện và khả năng truy vết.

2. Từ quy tắc sang test

Mỗi test tạo đúng trạng thái, gọi thao tác và xác minh quyền thật. Nếu UI ẩn nút, vẫn phải kiểm tra API trong môi trường được phép. Dữ liệu không hợp lệ như khóa đã gỡ không chỉ là một giá trị Free=false thông thường; nó có thể thuộc quy tắc tồn tại tài nguyên riêng.

3. Giảm tổ hợp có trách nhiệm

Ba điều kiện nhị phân tạo tối đa tám tổ hợp, trước khi loại các trường hợp bất khả thi. Pairwise nhắm tương tác từng cặp, không chứng minh bao phủ lỗi cần ba điều kiện. Tài liệu cần nêu cách chọn và phần bỏ lại.

4. Bài tập có đáp án

Giảm giá khi mã hợp lệ và tổng tiền ít nhất 500: cả hai đúng mới giảm. Tại biên 499/500/501, bổ sung kiểm tra giá trị biên bên cạnh bảng quyết định. Đừng tự dùng 499,99 nếu hợp đồng đang quy định đơn vị nguyên; xác nhận đơn vị tiền và cách làm tròn trước.

> **Bài tập vận dụng bổ sung**
>
> Lập bảng cho giảm giá chỉ khi có mã hợp lệ và tổng tiền đạt ngưỡng. Viết test cho bốn tổ hợp, thêm tình huống mã hết hạn theo yêu cầu cụ thể.

Hướng giải bài tập bổ sung

Hai điều kiện nhị phân có bốn tổ hợp trước khi rút gọn. Với phép AND, chỉ tổ hợp cả hai đúng được giảm giá.

Câu hỏi tự kiểm tra

1. Bảng quyết định phù hợp khi nào?
2. Hai điều kiện đúng/sai độc lập có bao nhiêu tổ hợp?
3. Với giảm giá khi A AND B, trường hợp nào được giảm?
4. Tổ hợp không thể xảy ra nên được xử lý thế nào?
5. Pairwise có bảo đảm tìm mọi lỗi ba điều kiện tương tác không?

Gợi ý đáp án và điểm cần nhớ

- Kết quả phụ thuộc tổ hợp nhiều điều kiện. Bảng nối các tổ hợp điều kiện với hành động.
- 4. Mỗi điều kiện có hai giá trị nên có 2 nhân 2 tổ hợp.
- A và B đều đúng. AND yêu cầu cả hai điều kiện thỏa.
- Ghi rõ lý do loại trừ theo yêu cầu. Cần minh bạch giả định và giới hạn của bảng.
- Không. Bao phủ cặp không tương đương bao phủ mọi bộ ba.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Bảng quyết định và tổ hợp điều kiện — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: học viên muốn mua khóa phụ thuộc đăng nhập, trạng thái mở bán, giá và ghi danh hiện tại. Liệt kê điều kiện và kết quả cho tổ hợp quan trọng, phát hiện quy tắc mâu thuẫn hoặc chưa định nghĩa.

Tự luyện: bảng gồm khóa miễn phí/có phí, đã/chưa ghi danh và đang/không mở bán. **Hướng giải:** chọn ca kiểm tra mỗi quy tắc; không suy pairwise bảo đảm tìm mọi lỗi tương tác ba điều kiện. Khi số tổ hợp lớn, nêu lý do chọn và rủi ro tổ hợp chưa thử.

Nguồn học tiếp

https://docs.pytest.org/en/stable/how-to/assert.html

Kiểm thử chuyển trạng thái

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Kiểm thử phần mềm nhập môn.
- Chương 4: Thiết kế kiểm thử theo hành vi.
- Các bài trong chương: Bảng quyết định và tổ hợp điều kiện; Kiểm thử chuyển trạng thái.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Mô hình hóa vòng đời nghiệp vụ và kiểm tra chuyển trạng thái hợp lệ, không hợp lệ.

Kiến thức nền

Mô hình trạng thái gồm trạng thái, sự kiện và chuyển tiếp. Kết quả của một hành động có thể phụ thuộc trạng thái trước đó. Ví dụ quiz đang làm, đã nộp hoặc hết thời gian không nên xử lý thao tác nộp giống nhau trong mọi trường hợp.

Bảng chuyển tiếp mô tả trạng thái đầu, hành động, trạng thái cuối và phản hồi. Ngoài đường hợp lệ, kiểm tra chuyển tiếp bị cấm như sửa câu trả lời sau khi đã nộp. Bao phủ trạng thái không đồng nghĩa bao phủ mọi chuyển tiếp giữa chúng.

Một lần chạy cần điều kiện khởi tạo rõ ràng và khả năng khôi phục dữ liệu. Với tác vụ bất đồng bộ, trạng thái giao diện và trạng thái server có thể cập nhật ở các thời điểm khác nhau; cần kiểm tra trạng thái có thẩm quyền. Không coi một nút bị ẩn là bằng chứng server đã ngăn hành động trái phép.

1. Mô hình hóa một lượt quiz

```
Chưa bắt đầu --mở hợp lệ--> Đang làm --nộp hợp lệ--> Đã nộp
Đang làm --hết thời gian--> Trạng thái theo quy tắc hệ thống
Đã nộp --sửa câu trả lời--> Từ chối, không đổi kết quả đã lưu
```

Không tự chọn “hết giờ luôn tự nộp” nếu chưa có yêu cầu. Phải xác nhận cách server xử lý và chỉ rõ người có thẩm quyền tính thời gian.

2. Trạng thái khác với nhãn giao diện

Một nút có chữ "đã nộp" không chứng minh dữ liệu đã commit. Test phải quan sát trạng thái được server xác nhận, kết quả và các tác dụng phụ. Cần xử lý mạng mất phản hồi sau khi server đã lưu: gửi lại có thể không được tạo kết quả thứ hai theo hợp đồng.

3. Bao phủ và chuỗi sự kiện

Test từng chuyển tiếp hợp lệ, chuyển tiếp bị cấm và chuỗi có ý nghĩa. Bao phủ mọi trạng thái chỉ cho biết đã tới các nút, không bảo đảm đã đi qua mọi cạnh. Thêm điều kiện dữ liệu như hết lượt, lượt đang làm hoặc quá giờ để phân biệt quy tắc.

4. Mẫu test và lời giải

Tạo attempt lab đang làm, nộp thành công, giữ result_id, gửi sửa câu trả lời sau nộp. Mong đợi từ chối theo API và hash kết quả cũ không đổi. Test khác mô phỏng request nộp lặp, xác nhận số kết quả theo hợp đồng. Không chỉnh đồng hồ hệ thống thật để thử hết giờ; dùng cơ chế thời gian kiểm soát được trong test.

Nộp bảng trạng thái đầu, sự kiện, guard, trạng thái cuối và dữ liệu cần giữ. Đánh dấu trạng thái chưa thử thay vì ghi pass theo sơ đồ.

> **Bài tập vận dụng bổ sung**
>
> Vẽ trạng thái của đơn hàng pending, paid, cancelled theo hợp đồng hệ thống. Viết ba chuyển tiếp hợp lệ và hai chuyển tiếp bị cấm; không dùng thanh toán thật.

Hướng giải bài tập bổ sung

Nêu rõ sự kiện dẫn tới chuyển trạng thái và kiểm tra cả dữ liệu server. Yêu cầu nộp trùng cần có kết quả mong đợi cụ thể.

Câu hỏi tự kiểm tra

1. Kết quả thao tác trong hệ thống có trạng thái phụ thuộc gì?
2. Bao phủ mọi trạng thái có bằng bao phủ mọi chuyển tiếp không?
3. Ẩn nút nộp sau hoàn tất đã chứng minh server chặn nộp lại chưa?
4. Test chuyển tiếp cần ghi thông tin nào?
5. Với chức năng bất đồng bộ nên đối chiếu thêm gì?

Gợi ý đáp án và điểm cần nhớ

- Trạng thái trước và sự kiện. Cùng sự kiện có thể được xử lý khác ở mỗi trạng thái.
- Không. Các đường nối trạng thái là mục tiêu bao phủ riêng.
- Chưa. Phải kiểm tra quy tắc tại API hoặc server.

- Trạng thái đầu, sự kiện và kết quả mong đợi. Các thành phần mô tả được luồng cần xác minh.
- Trạng thái server có thẩm quyền. Giao diện có thể cập nhật trước hoặc sau dữ liệu thực.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Kiểm thử chuyển trạng thái — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: nộp cùng token hai lần tạo hai kết quả. Mô hình lượt làm gồm mở, đang làm, nộp, hết giờ; mô hình đơn hàng có pending/completed/failed/refunded theo nghiệp vụ. Kiểm tra cả chuyển hợp lệ và bị cấm.

Tự luyện: thử nộp lại lượt đã hoàn tất và mở lại lượt đang làm. **Hướng giải:** kết quả chỉ lưu một lần; tải lại không tự tạo lượt mới; kiểm tra số bản ghi và quyền sau thao tác. Chú ý callback lặp, request đến chậm và thao tác ở nhiều tab, không chỉ đường thành công một tab.

Nguồn học tiếp

https://docs.pytest.org/en/stable/how-to/assert.html

Kiểm thử API và phân quyền

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Kiểm thử phần mềm nhập môn.
- Chương 5: Kiểm thử dữ liệu và dịch vụ.
- Các bài trong chương: Kiểm thử API và phân quyền; Kiểm thử cơ sở dữ liệu và tính toàn vẹn.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Thiết kế ca kiểm thử hợp đồng HTTP và quyền truy cập theo tài khoản.

Kiến thức nền

API test cần kiểm tra phương thức, đường dẫn, tham số, kiểu dữ liệu, mã trạng thái và nội dung phản hồi theo hợp đồng. Mã 200 không đủ chứng minh đúng: cần xác nhận dữ liệu, tác dụng phụ và quyền. Lỗi có thể nằm ở nghiệp vụ dù phản hồi vẫn là JSON hợp lệ.

Phân biệt xác thực danh tính với phân quyền tài nguyên. Kiểm tra người chưa đăng nhập, đúng chủ sở hữu, người khác và vai trò quản trị theo yêu cầu. Thay ID tài nguyên là tình huống kiểm tra quyền ở mức đối tượng quan trọng; chỉ thực hiện trên dữ liệu lab được phép.

Test đầu vào gồm thiếu trường, sai kiểu, giá trị biên và tài nguyên không tồn tại. Không đưa token vào báo cáo hoặc nguồn mã. Tạo dữ liệu test riêng và dọn trong phạm vi đó. Các mã phản hồi cụ thể phụ thuộc hợp đồng API, không suy ra mọi lỗi đều phải cùng một mã.

1. Hợp đồng request và response

Ghi method, path, headers cần thiết, schema body, trạng thái và schema response. Test API không chỉ kiểm tra “có JSON”: một JSON trả danh sách tài nguyên của người khác vẫn là sai quyền. Không ghi token vào fixture công khai hoặc ảnh minh chứng.

2. Mẫu yêu cầu minh họa

```
POST /api/enrollments
Authorization: Bearer <TOKEN_TEST_KHÔNG_CÔNG_KHAI>
Content-Type: application/json

{"course_id": 513}
```

Đây là mẫu minh họa; trước khi chạy phải đối chiếu endpoint và trường thực tế của môi trường thử. Không tự gửi hàng loạt tới production vì thấy một URL trong tài liệu.

3. Ma trận tài khoản và tài nguyên

```
Người gọi       Tài nguyên                 Kỳ vọng
Chưa xác thực   Bài làm riêng             Từ chối theo hợp đồng
U1              Bài làm U1                 Cho phép
U1              Bài làm U2                 Từ chối, không lộ nội dung
Giảng viên      Bài làm khóa mình phụ trách Theo quy tắc vai trò
```

Kiểm tra cả mã phản hồi lẫn nội dung bị rò rỉ. Trường hợp tài nguyên không tồn tại khác tài nguyên tồn tại nhưng không được phép, dù ứng dụng có thể cố ý dùng cùng phản hồi để giảm tiết lộ thông tin.

4. Bài tập và lời giải

Với API tạo ghi danh, viết test thiếu trường, chuỗi sai kiểu, khóa không tồn tại, lặp và đồng thời. Xác minh không tạo thanh toán với khóa miễn phí, không tạo dữ liệu khi bị từ chối. Hợp đồng có thể chấp nhận một số phép chuyển kiểu nên không tự coi mọi chuỗi số là lỗi nếu chưa xác nhận yêu cầu.

> **Bài tập vận dụng bổ sung**
>
> Lập năm test cho API đăng ký: hợp lệ, khóa không tồn tại, thiếu xác thực, đăng ký trùng và ID sai kiểu. Nêu điều kiện dữ liệu trước/sau, dùng môi trường thử nghiệm.

Hướng giải bài tập bổ sung

Kiểm tra đồng thời mã, schema, quyền và tác dụng dữ liệu. Không ghi token thật vào minh chứng.

Câu hỏi tự kiểm tra

1. API trả 200 đã đủ kết luận đúng nghiệp vụ chưa?
2. Xác thực trả lời câu hỏi nào?
3. Phân quyền trả lời câu hỏi nào?
4. Đổi ID sang tài nguyên của tài khoản khác nhằm kiểm tra gì?
5. Token test nên được quản lý thế nào?

Gợi ý đáp án và điểm cần nhớ

- Chưa, cần kiểm tra nội dung và dữ liệu. Thành công HTTP không bảo đảm kết quả nghiệp vụ đúng.
- Người gọi là ai. Authentication xác định danh tính.
- Người gọi được phép truy cập gì. Authorization áp dụng quyền trên thao tác và tài nguyên.

- Quyền ở mức đối tượng. Server phải kiểm tra quyền với tài nguyên được yêu cầu.
- Giữ bí mật và không đưa vào báo cáo. Token là thông tin xác thực cần bảo vệ.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Kiểm thử API và phân quyền — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: UI ẩn nút nhưng API vẫn cho người khác sửa khóa. Kiểm tra theo vai trò và theo tài nguyên thuộc ai, không chỉ đăng nhập/chưa đăng nhập.

Tự luyện: dùng tài khoản A tạo tài nguyên lab; B thử đọc/sửa/xóa theo hợp đồng. **Hướng giải:** mã từ chối phù hợp và dữ liệu không đổi; một số API dùng 404 để không tiết lộ sự tồn tại. Với AI Tutor, HTTP 200 có thể chứa lỗi LLM hoặc câu từ chối: chấm nội dung, nguồn và phạm vi riêng, không gọi mọi HTTP 200 là câu trả lời tốt.

Nguồn học tiếp

https://docs.pytest.org/en/stable/how-to/assert.html

Kiểm thử cơ sở dữ liệu và tính toàn vẹn

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Kiểm thử phần mềm nhập môn.
- Chương 5: Kiểm thử dữ liệu và dịch vụ.
- Các bài trong chương: Kiểm thử API và phân quyền; Kiểm thử cơ sở dữ liệu và tính toàn vẹn.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

Mục tiêu bài học

Đối chiếu thao tác ứng dụng với ràng buộc dữ liệu và rollback trong môi trường test.

Kiến thức nền

Kiểm thử dữ liệu xác nhận bản ghi được tạo, cập nhật hoặc giữ nguyên đúng yêu cầu. Kiểm tra khóa ngoại, tính duy nhất, giá trị NULL và ràng buộc miền. Với ghi danh, một cặp học viên–khóa có thể cần duy nhất; chỉ chặn bấm hai lần ở giao diện không đủ xử lý hai yêu cầu đồng thời.

Transaction nhóm thay đổi thành đơn vị công việc. Nếu một bước thất bại, rollback giúp tránh trạng thái một phần theo thiết kế giao dịch. Test cần xác minh trạng thái trước/sau, không chỉ thấy thông báo lỗi là cho rằng dữ liệu chưa đổi.

Dùng database test hoặc bản sao đã được phép, đánh dấu dữ liệu tạo cho test và dọn đúng phạm vi. Không chạy lệnh xóa hàng loạt trên dữ liệu học viên thật. Cần tính cả số bản ghi, liên kết và trường quan trọng; một phép COUNT không chứng minh mọi trường đã đúng.

1. Bằng chứng dữ liệu cần kiểm tra

Đếm bản ghi chỉ là một phần. Xác minh đúng user_id, course_id, trạng thái và trường thời gian nếu liên quan. Một bản ghi tồn tại nhưng gắn sai người dùng là lỗi dù COUNT tăng đúng một. Không lấy nội dung toast làm bằng chứng duy nhất cho commit.

2. Schema lab minh họa

```
CREATE TABLE lab_enrollments (
    id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    UNIQUE (user_id, course_id)
);
```

Chỉ tạo bảng này trong database test. Cú pháp ID tự tăng và khóa ngoại phụ thuộc hệ quản trị; ví dụ tối thiểu không đại diện toàn schema của ứng dụng. Ghi lại ràng buộc thực tế trước khi viết kỳ vọng.

3. Test giao dịch nhiều bước

Tạo bản ghi cha, tạo bản ghi liên quan, gây lỗi có kiểm soát trước commit rồi kiểm tra rollback. Một thư viện có thể tự commit từng helper nên việc đặt hai lời gọi cạnh nhau không bảo đảm chúng nằm trong một transaction. Phải kiểm tra ranh giới giao dịch thật.

4. Đồng thời và dọn dữ liệu

Hai request đều kiểm tra "chưa có" trước khi insert có thể cùng vượt bước kiểm tra. Ràng buộc unique ở database là lớp bảo vệ cuối, còn server cần xử lý lỗi để trả phản hồi có nghĩa. Dọn test theo ID đã tạo, không DELETE toàn bảng hoặc quét theo tên người dùng thật.

5. Bài tập có tiêu chí

Thiết kế test trùng, NULL, khóa ngoại sai và rollback; nêu môi trường test, dữ liệu trước/sau và phép query đọc để xác nhận. Đạt khi bằng chứng phân biệt "request lỗi nhưng đã ghi một phần" với "giao dịch được hoàn tác đúng".

> **Bài tập vận dụng bổ sung**
>
> Thiết kế test chèn trùng, khóa ngoại không tồn tại và lỗi ở bước hai của giao dịch. Ghi trạng thái mong đợi của cả hai bảng, thực hiện trên database test.

Hướng giải bài tập bổ sung

Sau rollback đúng thiết kế, không còn thay đổi một phần trong giao dịch đó. Việc dọn test phải dựa vào ID test đã tạo.

Câu hỏi tự kiểm tra

1. Ràng buộc unique thường dùng để làm gì?
2. Khóa ngoại hỗ trợ điều gì?
3. Chỉ khóa nút đăng ký có xử lý chắc chắn yêu cầu đồng thời không?
4. Rollback được dùng để làm gì?
5. Môi trường nào phù hợp cho test ghi/xóa dữ liệu?

Gợi ý đáp án và điểm cần nhớ

- Ngăn giá trị hoặc tổ hợp khóa bị trùng theo quy tắc. Unique biểu diễn tính duy nhất ở mức database.
- Toàn vẹn tham chiếu. Khóa ngoại liên kết tới bản ghi hợp lệ theo ràng buộc.

- Không. Hai client hoặc hai request vẫn có thể đến server.
- Hoàn tác thay đổi chưa commit của giao dịch. Rollback xử lý thay đổi trong phạm vi transaction.
- Database test hoặc bản sao được phép. Dữ liệu thử cần được cô lập và kiểm soát.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Kiểm thử cơ sở dữ liệu và tính toàn vẹn — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: API báo thất bại nhưng vẫn ghi một nửa giao dịch. Kiểm tra tính nguyên tử, khóa ngoại, unique và dữ liệu sau đường lỗi. Một số thử nghiệm cần transaction hoặc bản sao database lab.

Tự luyện: đăng ký trùng, xóa khóa có bài làm và hai request đồng thời trên dữ liệu riêng. **Hướng giải:** không đếm chỉ response; xác nhận số dòng và quan hệ. Chỉ dọn những id do lượt test tạo, không dùng DELETE theo từ khóa rộng trên dữ liệu có thể thuộc người khác.

Nguồn học tiếp

https://docs.pytest.org/en/stable/how-to/assert.html

Hiệu năng, khả dụng và tương thích

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Kiểm thử phần mềm nhập môn.
- Chương 6: Chất lượng và thực hành tổng hợp.
- Các bài trong chương: Hiệu năng, khả dụng và tương thích; Dự án nhỏ: kế hoạch kiểm thử và tự động hóa.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Viết tiêu chí đo chất lượng ngoài chức năng trong phạm vi lab.

Kiến thức nền

Kiểm thử phi chức năng xem đặc tính như hiệu năng, khả dụng, khả năng tiếp cận và tương thích. Một chức năng đúng kết quả nhưng chậm hoặc không thao tác được bằng bàn phím vẫn có thể không đáp ứng nhu cầu người dùng.

Hiệu năng cần mô hình tải, dữ liệu, môi trường, thước đo và ngưỡng đã thống nhất. Thời gian phản hồi, thông lượng và tỉ lệ lỗi là các thước đo khác nhau. Percentile như p95 mô tả phần lớn mẫu tốt hơn một trung bình đơn lẻ khi có đuôi chậm. Đo tải chỉ trên hệ thống được phép và có giới hạn để tránh ảnh hưởng người dùng.

Kiểm tra giao diện với tên dài, màn hình hẹp, phóng to và thao tác bàn phím. Thông báo cần rõ, không chỉ dùng màu để truyền trạng thái. Kiểm tra tương thích theo tập trình duyệt/thiết bị được hỗ trợ; không tuyên bố đã thử mọi nền tảng nếu chỉ có một môi trường.

1. Viết mục tiêu đo được

Thay "trang nhanh" bằng một tiêu chí ví dụ: trên môi trường lab đã nêu, dữ liệu 100 khóa, dưới mô hình tải được phép, đo p95 thời gian API và tỉ lệ lỗi theo ngưỡng nhóm thống nhất. Đây là mẫu đặc tả, không phải kết quả website hiện tại hay ngưỡng chuẩn cho mọi hệ thống.

2. Đọc số liệu có ngữ cảnh

Ghi thời gian làm nóng, số mẫu, phân bố độ trễ và tỉ lệ lỗi. Không so phiên bản A trên máy rảnh với B trên máy đang bận rồi quy mọi khác biệt cho mã nguồn. Timeout và lỗi HTTP cần được tính, không loại khỏi báo cáo để làm trung bình đẹp hơn.

3. Checklist thao tác giao diện

- Dùng Tab tới nút đăng ký, có focus rõ và kích hoạt bằng bàn phím.
- Thông báo đọc được, có văn bản chứ không chỉ màu; có nút đóng.
- Tên khóa dài không che số liệu hoặc đẩy nút ra ngoài màn hình.
- Ở màn hình hẹp và khi phóng to, không mất nội dung quan trọng.
- Mạng chậm có trạng thái chờ và ngăn bấm lặp phù hợp.
- Lỗi API không hiển thị như chưa có dữ liệu.

Một checklist thủ công hữu ích nhưng không tự chứng nhận đạt toàn bộ tiêu chuẩn tiếp cận.

4. Bài tập và đáp án định hướng

Soạn ma trận ba kích thước khung và hai trình duyệt được hỗ trợ. Chọn một luồng chính kiểm thử ở cả sáu tổ hợp, ghi môi trường. Tạo tên dài và thông báo lỗi dài trong dữ liệu lab. Đáp án cần có ảnh và nhận xét cụ thể, không chỉ "đẹp"; ghi giới hạn khi chưa có thiết bị thật hoặc công nghệ hỗ trợ để thử.

> **Bài tập vận dụng bổ sung**
>
> Viết checklist 8 mục cho trang khóa học: ít nhất hai mục về bàn phím, hai về màn hình hẹp, hai về nội dung dài và hai về hiệu năng có điều kiện đo.

Hướng giải bài tập bổ sung

Ghi rõ môi trường và tiêu chí, không dùng câu "rất nhanh" hay "đẹp" làm kết quả mong đợi duy nhất. p95 không phải giá trị chậm nhất tuyệt đối.

Câu hỏi tự kiểm tra

1. Kiểm thử phi chức năng xem xét gì?
2. Để tiêu chí hiệu năng kiểm chứng được cần gì?
3. p95 thời gian phản hồi thường diễn đạt điều gì?
4. Vì sao không nên chỉ dùng màu báo lỗi?
5. Test tải nên thực hiện ở đâu?

Gợi ý đáp án và điểm cần nhớ

- Đặc tính chất lượng như hiệu năng và khả dụng. Phi chức năng đánh giá cách hệ thống đáp ứng ngoài kết quả nghiệp vụ.
- Tải, môi trường, thước đo và ngưỡng. Các điều kiện đo làm kết quả có ý nghĩa.
- Khoảng 95% mẫu không vượt ngưỡng đó. Percentile mô tả phân bố mẫu đo.
- Người dùng cần tín hiệu văn bản hoặc biểu tượng dễ hiểu. Trạng thái cần được truyền bằng nhiều dấu hiệu có thể tiếp cận.

- Hệ thống được phép với giới hạn đã xác định. Hoạt động tải cần phạm vi và kiểm soát tác động.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Hiệu năng, khả dụng và tương thích — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: "trang nhanh" không có số đo. Xác định môi trường, tải, thời gian đầu/cuối và phân vị; p95 không phải chậm nhất. Kiểm tra khả dụng gồm bàn phím, thông báo lỗi và màn hình hẹp.

Tự luyện: đo một tác vụ 20 lần trong điều kiện ghi rõ và liệt kê các trạng thái loading/error/rỗng.
Hướng giải: báo cả mẫu chậm, không chỉ trung bình; phân biệt thời gian chờ cố ý trong script với độ trễ ứng dụng. Không suy tải đồng thời lớn từ một người chạy local hoặc suy mọi trình duyệt từ một lần thử Chromium.

Nguồn học tiếp

https://docs.pytest.org/en/stable/how-to/assert.html

Dự án nhỏ: kế hoạch kiểm thử và tự động hóa

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Kiểm thử phần mềm nhập môn.
- Chương 6: Chất lượng và thực hành tổng hợp.
- Các bài trong chương: Hiệu năng, khả dụng và tương thích; Dự án nhỏ: kế hoạch kiểm thử và tự động hóa.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

Mục tiêu bài học

Tạo gói kiểm thử có thể chạy lại cho một chức năng học trực tuyến.

Kiến thức nền

Tự động hóa phù hợp với ca lặp lại, kết quả mong đợi ổn định và giá trị phát hiện lỗi đủ lớn so với chi phí bảo trì. Không tự động hóa mọi thao tác chỉ vì có công cụ. Tách unit test nhanh, API/integration và một số luồng giao diện quan trọng theo nhu cầu sản phẩm.

Một test tốt có chuẩn bị, hành động và kiểm tra kết quả cụ thể. Các test nên độc lập, dữ liệu có thể tái tạo và dọn trong phạm vi đã tạo. Khi test chập chờn, tìm nguyên nhân như chờ bất đồng bộ hoặc phụ thuộc thứ tự, không đơn giản chạy lại tới khi xanh rồi bỏ qua.

Dự án chọn chức năng ghi danh hoặc nộp quiz: lập phạm vi, rủi ro, test case, dữ liệu, báo cáo lỗi và kết quả hồi quy. Phân biệt chức năng chưa kiểm tra với test đã qua. Tài liệu kết thúc phải nêu rủi ro còn lại; quiz không tự xác nhận bộ test thực tế đã được triển khai.

1. Phạm vi gói bàn giao

Chọn đăng ký khóa miễn phí làm tính năng mục tiêu. Tài liệu cần mục tiêu, phạm vi không làm, rủi ro, môi trường, dữ liệu, trách nhiệm, điều kiện bắt đầu/kết thúc và cách báo cáo. Chỉ tự động hóa các kiểm tra có kỳ vọng rõ, không tạo assertion theo dữ liệu đang trả về để test luôn xanh.

2. Ví dụ test độc lập không gọi hệ thống thật

```
def can_open_lesson(is_free, enrolled):
    return is_free or enrolled

def test_guest_cannot_open_restricted_lesson():
    assert can_open_lesson(False, False) is False
```

```
def test_enrolled_can_open_restricted_lesson():
    assert can_open_lesson(False, True) is True

def test_free_lesson_is_open():
    assert can_open_lesson(True, False) is True
```

Đây là mô hình luật đơn giản phục vụ học cách viết test, không phải toàn bộ hàm phân quyền của LMS. Trong dự án thật còn trạng thái khóa, tài khoản và vai trò; cần test implementation thật qua interface phù hợp.

3. Từ ví dụ tới kiểm tra tích hợp

Tạo user/course lab trong fixture, gọi API thực ở môi trường test, xác nhận phản hồi và dữ liệu. Mỗi test phải tự có điều kiện trước và dọn dữ liệu của nó. Test UI chờ điều kiện quan sát được thay vì sleep cố định dài. Khi flaky, thu trace và sửa nguyên nhân, không chỉ tăng timeout vô hạn.

4. Rubric tự đánh giá

Chấm yêu cầu có truy vết, bao phủ rủi ro, dữ liệu cô lập, assertions đúng, khả năng chạy lại và báo cáo trung thực. Nộp 12 test thiết kế, ba test tự động, kết quả thực chạy và hai báo cáo lỗi nếu tìm thấy; nếu dùng lỗi giả định phải gắn nhãn giả định. Không bịa lỗi hay kết quả chỉ để đủ số lượng bài nộp.

> **Bài tập vận dụng bổ sung**
>
> Nộp kế hoạch một trang, 12 test gồm quyền/biên/trạng thái/dữ liệu, hai báo cáo lỗi mẫu ghi rõ là tình huống giả định và ba test tự động có thể chạy lại trên lab. Không tạo lỗi hoặc kết quả giả vào hệ thống thật.

Hướng giải bài tập bổ sung

Đánh giá theo tiêu chí: truy vết yêu cầu, kết quả mong đợi rõ, dữ liệu độc lập, kiểm tra có ý nghĩa, bằng chứng chạy và giới hạn. Không tính số lượng test là thước đo duy nhất.

Câu hỏi tự kiểm tra

1. Ca nào thường phù hợp tự động hóa?
2. Ba phần cơ bản của test thường là gì?
3. Test độc lập có lợi ích gì?
4. Test chập chờn nên được xử lý thế nào?
5. Báo cáo kết thúc kiểm thử cần nêu gì?

Gợi ý đáp án và điểm cần nhớ

- Ca lặp lại có kỳ vọng ổn định và giá trị rõ. Cần cân nhắc giá trị và bảo trì.

- Chuẩn bị, hành động, kiểm tra. Test cần thiết lập trạng thái, thực hiện và xác minh.
- Giảm phụ thuộc thứ tự chạy. Có thể chạy lại riêng và theo thứ tự khác.
- Tìm nguyên nhân dữ liệu, thời gian hoặc phụ thuộc. Flaky test làm giảm độ tin cậy của phản hồi.
- Phạm vi, kết quả và rủi ro còn lại. Người đọc cần hiểu cả bằng chứng lẫn giới hạn.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Dự án nhỏ: kế hoạch kiểm thử và tự động hóa — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Bài thực hành: chọn ghi danh hoặc nộp quiz trong lab, tạo 12 ca có quyền/biên/trạng thái, ba test tự động có assertion và báo cáo kết quả. Tách dữ liệu chuẩn bị, hành động, xác minh và cleanup.

```
def test_valid_boundary():
    assert valid_score(0)
    assert valid_score(10)
```

Đoạn trên chạy sau định nghĩa valid_score của bài biên; không phải test kết nối vào hệ thống thật.

Hướng giải: test tự động phải phát hiện được hành vi sai có ý nghĩa, không chỉ khẳng định dữ liệu vừa tự tạo bằng đúng đoạn code tương tự. Ghi rõ test qua, fail, chưa chạy và giới hạn. Kết quả mẫu không phải bằng chứng tác vụ đó đã được chạy thật.

Nguồn học tiếp

https://docs.pytest.org/en/stable/how-to/assert.html